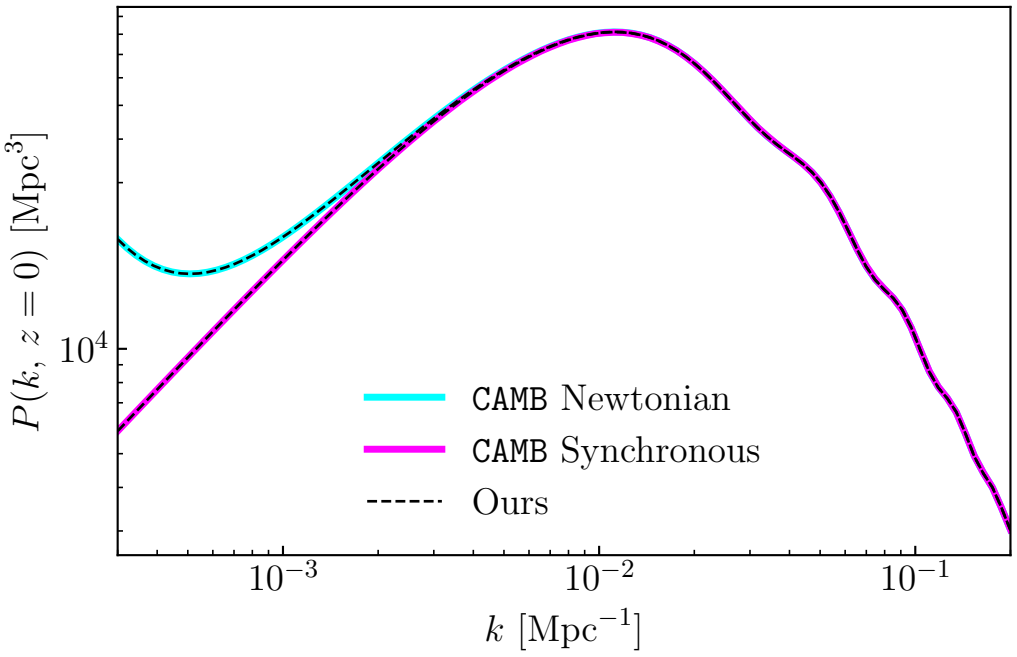
Boltzmann Code - Fun things to do

The goal of today's lab is to show you some fun things you can now do with your lovingly hand-crafted Boltzmann code.

Linear Matter Power Spectrum:



We can get the linear matter power spectrum by applying the transfer functions we've computed with our code to the primordial curvature power spectrum

$$P_m(k) = P_{\mathcal{R}}(k)[T_m(k)]^2 \text{ where } \frac{k^3}{2\pi^2}P_{\mathcal{R}}(k) = \Delta^2_{\mathcal{R}}(k) = A_s\left(\frac{k}{0.05\,\text{Mpc}^{-1}}\right)^{n_s-1}.$$

(Note: what we've been calling $\delta_a(k,\eta)$ in your Boltzmann code are really the transfer functions $T_a(k,\eta)$) The total matter density contrast is given by

$$\delta_m = \frac{\rho_c\delta_c + \rho_b\delta_b}{\rho_c + \rho_b}.$$

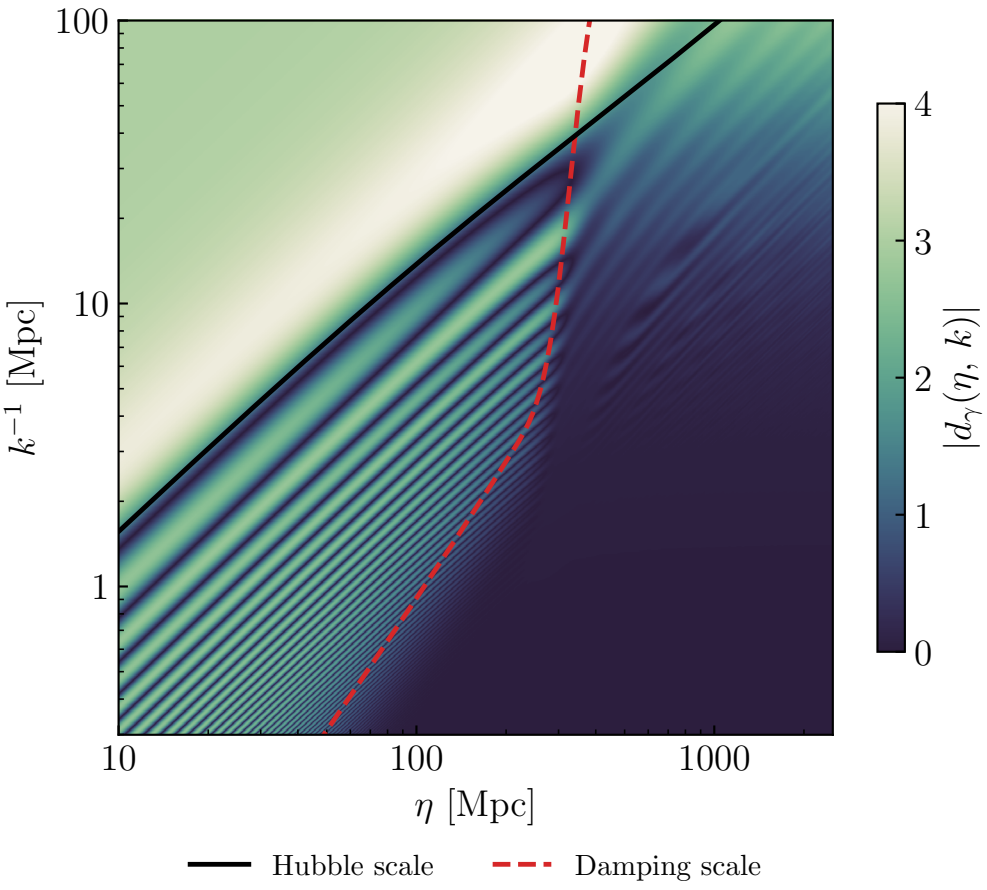
We've been working in Newtonian gauge where

$$\delta_c^N = d_c + 3\Psi, \quad \delta_b^N = d_b + 3\Psi,$$

but also compute things in synchronous gauge where in the CDM rest frame

$$\delta_c^S = \delta_c^N + 3\mathcal{H}u_c, \quad \delta_b^S = \delta_b^N + 3\mathcal{H}u_c.$$

Photon transfer function Basically recreate Fig.(7.10) of Baumann but using only code you wrote yourself instead of `CLASS` or `CAMB` like he does.

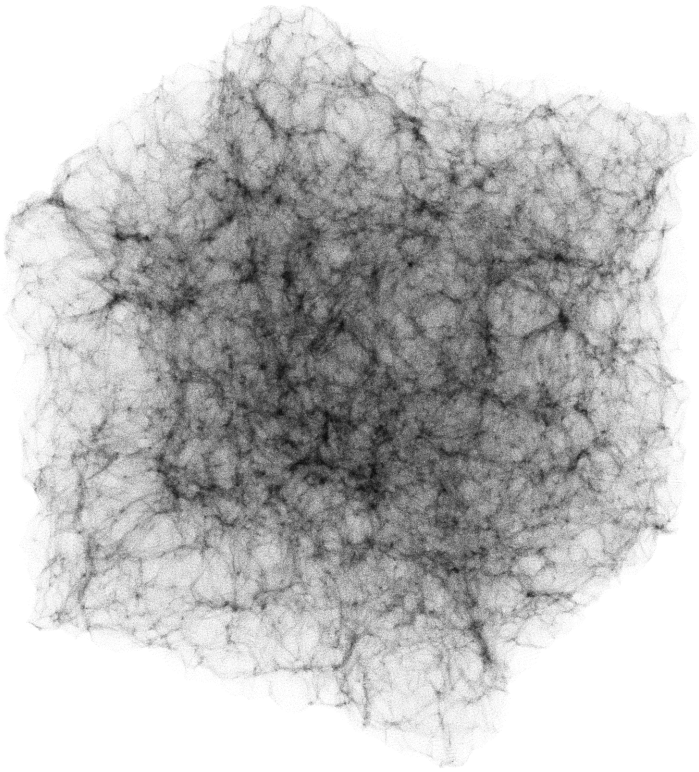


I think it's fun to stare at this and see the whole picture of modes entering the horizon, damping, sound waves, etc. Look at Baumann Eq.(C.26) for damping

scale. At this point we really start getting punished for writing our code in plain Python because it is very slow... start by debugging with a much lower resolution version and the once the low-res version looks right you can set the slow code running overnight and wake up to a nice high-res version.

Zeldovich approximation Once you have the power spectrum, you can generate realizations of perturbations by generating Gaussian random fields and multiplying them with the correct power spectrum. You can very easily evolve this forward with the Zeldovich approximation.

Zeldovich Approx, $z = 0$



I'll defer most of the details on the Zeldovich approximation to some old notes I have shared in `aux/HW07*` and probably some future lecture from Oliver. To generate GRFs with the correct $P(k)$ (for more details see my old handout)

```
1  LBOX = 512.0 # Mpc
2  NGRID = 128
3  dX = LBOX/NGRID
4
5  #... some code that gets power spectrum
6  # and interpolates it in log-log space
7  #   -> f_logPk
8
9  _k = 2.0 * np.pi * np.fft.fftfreq(NGRID, d=dX)
10 kmesh = np.stack(np.meshgrid(_k, _k, _k, indexing='ij')) # (3, N, N, N)
11 k_abs = np.sqrt(_k[:, None, None]**2 + _k[None, :, None]**2 + _k[None,
       None, :]**2)
12
13 delta_q_0 = np.random.normal(size=(NGRID, NGRID, NGRID))
14 delta_q_0 /= np.sqrt(dX**3)
15
16 delta_k_0 = np.fft.fftn(delta_q_0) * dX**3
17 delta_k_0 *= np.sqrt(np.exp(f_logPk(np.log(k_abs))))
18 delta_k_0[0, 0, 0] = 0.0
19
20 delta_q_0 = np.real(np.fft.ifftn(delta_k_0)) / dX**3
```

At least once in your life you should look at the **numpy.fft** conventions in comparison to the usual Fourier conventions we use in cosmology and convince yourself the factors of dV on lines 16 and 20 belong. Something more subtle worth understanding once you get things working is why the factor of $1/\sqrt{dV}$ is there on line 14.